

UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas

"Plataforma web para la comercialización y revalorización de productos perecibles orientada al consumo sostenible en la provincia de Tacna"

Curso: Construcción de Software I

Docente: Mag. Ricardo Eduardo Valcárcel Alvarado

Integrantes:

Renzo Fernando Loyola Vilca ***(2021072615)***

Brant Antony Chata Choque ***(2020067577)***

Tacna – Perú

2026

1. INTRODUCCIÓN Y METODOLOGÍA

Este informe evalúa la trazabilidad del Sprint 2 de ÓrbiKa, planificado del 19 al 24 de octubre de 2026, cuyo objetivo es permitir la publicación y actualización de ofertas válidas, mostrar el descuento calculado con precisión decimal y ofrecer formularios utilizables en dispositivos móviles.

El análisis contrasta cuatro compromisos del Sprint Backlog —RF-03, RF-04, RF-19 y RNF-06— con la especificación del sistema, las decisiones de diseño documentadas, el código fuente y las pruebas registradas en el repositorio. Se distingue entre la existencia de código, las pruebas automatizadas locales y la aceptación en un entorno real de Supabase.

Alcance del Sprint 2

ID	Requisito	Prioridad	Objetivo de aceptación
RF-03	Publicar un producto perecible	Alta	Vendedor autenticado publica una oferta válida con nombre, categoría, precios, stock y vencimiento.
RF-04	Mostrar porcentaje de descuento	Media	El porcentaje coincide con la fórmula y se redondea a un decimal.
RF-19	Editar producto propio	Media	El propietario puede editar campos permitidos; otro vendedor no puede modificarlo.
RNF-06	Interfaz responsive	Media	Los flujos principales deben ser utilizables entre 320 px y 2560 px sin desplazamiento horizontal del contenido principal.

2. COMPROMISO DEL SPRINT 2 (SEGÚN EL SPRINT BACKLOG)

Campo	Contenido
Objetivo	Permitir publicar y actualizar ofertas válidas con precio transparente y formularios usables en móvil.
Compromiso	RF-03, RF-04, RF-19 y RNF-06.
Dependencia	Depende del Sprint 1: se requiere vendedor autenticado.
Entregable planificado	Flujo de publicación/edición y detalle con descuento calculado; registro de validación responsive.
Criterio de cumplimiento	Vendedor autenticado publica oferta válida; descuento coincide con la fórmula; otro vendedor no puede editar; contenido principal cabe desde 320 px.
Evidencia planificada	Capturas del formulario y catálogo, más resultados de validación en resoluciones definidas.
Definition of Done	Criterios aceptados; pruebas y resultados registrados; cambios integrados; sin defectos críticos abiertos; evidencia guardada sin credenciales ni datos personales reales.

Resumen ejecutivo de la verificación

Área	Resultado observado	Estado de trazabilidad
RF-03 Publicación	Formulario, validación de backend, control por rol y pruebas para datos inválidos están presentes en el código.	Implementado en código; aceptación remota pendiente.
RF-04 Descuento	Se corrigió el cálculo para conservar un decimal en frontend y vista SQL mediante migración 004.	Implementado en código; aplicación remota de migración pendiente.
RF-19 Edición	Se valida rol, propiedad, campos editables y conflicto de actualización.	Implementado en código; validación remota pendiente.
RNF-06 Responsive	Hay clases responsive y rejillas de una/dos columnas; no se encontró medición documentada de toda la banda 320–2560 px.	Parcialmente verificable; falta evidencia de viewport.

Interpretación: el estado “implementado en código” no equivale a “aceptado en producción”. Para cerrar el Sprint 2, el equipo debe conservar capturas y resultados reproducibles en un entorno configurado con Supabase real.

3. REQUERIMIENTO 1 — RF-03: PUBLICAR UN PRODUCTO PERECIBLE

Elemento	Descripción
SRS — definición del requisito	El vendedor publica un producto perecible para ofrecerlo a compradores, registrando la información comercial y de disponibilidad.
Caso de uso relacionado	CU-07: Publicar producto. El flujo requiere vendedor autorizado y una oferta válida. Las notas del SRS admiten fotografías JPG o PNG de hasta 5 MB.
Decisión / diseño	La publicación se separa entre presentación (formulario), lógica de negocio (validaciones y autorización) y persistencia mediante la API y Supabase/PostgreSQL.
Evidencia en frontend	frontend/src/pages/PublishProduct.jsx usa ProductForm y llama a marketplace.publicarProducto(datos). El formulario recoge nombre, descripción, categoría, precios, stock, fecha de vencimiento, zona, modalidad de entrega, peso e imagen.
Evidencia en backend	backend/src/routes/producto.routes.js protege POST / con requireAuth y requireRole('vendedor'). backend/src/services/producto.service.js vuelve a exigir vendedor activo y llama a validarProductoNuevo().
Validaciones identificadas	El backend rechaza precio actual superior al original, precio/stock no positivos, vencimiento igual o anterior a hoy, modalidad inválida y campos de texto que superen límites. La imagen del formulario acepta JPG/PNG hasta 5 MB.
Pruebas relacionadas	backend/test/validation/producto.validation.test.js cubre producto válido y rechazo de precio mayor, stock cero, vencimiento no válido y modalidad inválida. backend/test/services/producto.service.test.js verifica que un comprador no pueda publicar.
Criterio de aceptación	El código cubre la autorización y validación de datos. La publicación persistida, carga real de foto y aparición tras recargar el catálogo requieren confirmación con Supabase alojado.

Estado de desarrollo: implementado en código; aceptación remota pendiente

Observación: la interfaz permite una foto sugerida por URL o una imagen subida. La validación de formato y tamaño se ve en ProductForm.jsx; el funcionamiento de Storage alojado no queda demostrado solo con las pruebas locales.

4. REQUERIMIENTO 2 — RF-04: MOSTRAR EL PORCENTAJE DE DESCUENTO

Elemento	Descripción
SRS — definición del requisito	El comprador debe poder comparar el ahorro mediante un porcentaje de descuento calculado desde el precio original y el precio actual.
Regla de cálculo	Descuento (%) = ((precio original − precio actual) / precio original) × 100. La planificación exige redondear a un decimal y el precio original debe ser mayor que cero.
Hallazgo inicial del Sprint 2	La documentación orbika-app/docs/sprints-01-02.md indica que el descuento estaba redondeado a enteros, lo que no coincidía con el criterio decimal de la planificación.
Evidencia en frontend	frontend/src/pages/product.helpers.js implementa calcularAhorro(). Convierte precios a céntimos y redondea el porcentaje a un decimal. ProductForm.jsx muestra porcentaje y monto ahorrado.
Evidencia en base de datos	supabase/migrations/004_discount_precision.sql renombra el descuento entero previo y añade descuento_pct con redondeo a un decimal, preservando la vista y sus dependencias.
Pruebas relacionadas	frontend/src/pages/product.helpers.test.js prueba 42,9 % para precios 7 y 4; 33,3 % para 3 y 2; 0,1 % para 100 y 99,9; y 50,1 % para 80 y 39,96. La documentación técnica también registra verificación SQL de esos valores.
Criterio de aceptación	La lógica de frontend y la vista SQL contemplan el decimal. La coherencia completa con catálogo real depende de ejecutar la migración 004 en el entorno Supabase que utilice el equipo.

Estado de desarrollo: implementado en código; despliegue de migración por verificar

Observación: la fórmula debe ser consistente entre formulario, tarjeta del catálogo y base de datos. El descuento se calcula desde los dos precios; no es un campo independiente que se edite manualmente.

5. REQUERIMIENTO 3 — RF-19: EDITAR UN PRODUCTO PROPIO

Elemento	Descripción
SRS — definición del requisito	El vendedor puede actualizar su oferta para mantener vigentes el precio, stock, descripción y otros campos autorizados.
Caso de uso relacionado	CU-19: Editar producto. La modificación debe respetar la identidad del propietario y no permitir cambiar campos internos o de publicación que no estén autorizados.
Evidencia en frontend	frontend/src/pages/EditProduct.jsx obtiene el producto, carga los valores en ProductForm y llama a marketplace.editarProducto(id, datos). La categoría aparece deshabilitada durante la edición.
Evidencia en rutas	backend/src/routes/producto.routes.js protege PUT /:id mediante requireAuth y requireRole('vendedor').
Evidencia en servicio	backend/src/services/producto.service.js compara actual.vendedor_id con actor.id, rechaza productos ajenos y limita las propiedades que pueden actualizarse. La actualización incluye el stock anterior como condición para detectar conflictos concurrentes.
Campos y reglas	Se admiten campos permitidos como nombre, descripción, precios, stock, zona, modalidad e imagen. Categoría, fecha de vencimiento y peso de la unidad se conservan de la publicación original; campos internos como vendedor_id se rechazan.
Pruebas relacionadas	backend/test/services/producto.service.test.js verifica rechazo de campos internos, prohibición de editar productos ajenos, actualización de stock agotado y no reactivación de una publicación retirada. backend/test/validation/producto.validation.test.js comprueba campos no editables.
Criterio de aceptación	La autorización por propiedad y la lista de campos editables están cubiertas por código y pruebas. Se debe verificar en entorno real el cambio persistido y la respuesta de denegación al intentar editar con otra cuenta.

Estado de desarrollo: implementado en código; aceptación remota pendiente

Observación: la actualización condicional por stock anterior ayuda a evitar que una edición sobrescriba una modificación concurrente. Si el producto cambió, la API devuelve un conflicto y solicita recargar.

6. REQUERIMIENTO NO FUNCIONAL — RNF-06: INTERFAZ RESPONSIVE

Elemento	Descripción
SRS / Sprint Backlog	La interfaz debe permitir utilizar los flujos principales en teléfonos, tabletas y computadoras. La planificación establece un rango de 320 px a 2560 px sin desplazamiento horizontal del contenido principal.
Evidencia en publicación	frontend/src/pages/PublishProduct.jsx usa un contenedor fluido con padding horizontal y cambios de espaciado desde el breakpoint sm.
Evidencia en formulario	frontend/src/components/ProductForm.jsx usa rejillas grid-cols-1 sm:grid-cols-2 para campos, flex-col sm:flex-row para el bloque de demostración y espaciado adaptable.
Evidencia en edición	frontend/src/pages/EditProduct.jsx aplica contenedor max-w-2xl, padding horizontal y cambios de espaciado con sm:py-12.
Verificación disponible	La estructura de clases muestra intención responsive. La documentación de orbika-app/docs/sprints-01-02.md menciona pruebas de navegador, pero no aporta una tabla de resultados que certifique todas las resoluciones desde 320 px hasta 2560 px.
Criterio de aceptación	Para declarar cumplimiento total se requieren capturas y resultados de publicación/edición en resoluciones definidas, como 320, 375, 768, 1024, 1440 y 2560 px, confirmando ausencia de scroll horizontal en el contenido principal.

Estado de desarrollo: parcialmente verificable

Recomendación de evidencia: registrar cada viewport con la pantalla y el resultado de la comprobación. No se debe declarar el RNF-06 completamente aceptado basándose únicamente en la presencia de clases responsive.

7. VERIFICACIÓN DEL DEFINITION OF DONE DEL SPRINT 2

Criterio del DoD	Evaluación	Evidencia / observación
Criterios de aceptación cubiertos	Mayormente sí	RF-03 y RF-19 cuentan con validaciones y autorización en backend; RF-04 tiene cálculo decimal. RNF-06 requiere pruebas documentadas por viewport.
Pruebas y resultados registrados	Sí, según documentación local	orbika-app/docs/sprints-01-02.md registra 91 pruebas backend y 49 frontend aprobadas, además de compilación correcta y recorridos de navegador. Esos datos son un registro del repositorio, no una ejecución repetida en este informe.
Cambios integrados	Parcialmente verificable	Los cambios aparecen en el ZIP/revisión del repositorio. Para cerrar formalmente, adjuntar el commit o Pull Request correspondiente al sprint.
Sin defectos críticos abiertos	No verificable	No se encontró en las fuentes revisadas un registro formal de incidencias cerradas/abiertas que permita confirmar este criterio.
Evidencia guardada	Parcial	Existe documentación técnica de pruebas, pero no se adjunta una serie de capturas de resolución para el Sprint 2 ni resultados de aceptación en Supabase remoto.
Aceptación con servicios reales	Pendiente	El documento técnico indica que Auth/API se simulan en pruebas y que la aceptación alojada con Supabase está pendiente.

Resumen de pruebas reportadas en la documentación del repositorio

Comprobación	Resultado registrado	Alcance / límite
Backend, incluida validación y migraciones SQL	91 pruebas aprobadas	Ejecución local con servicios simulados y PostgreSQL en memoria (PGlite).
Frontend	49 pruebas aprobadas	Incluye cálculo de descuento no entero y empates de redondeo.
Compilación frontend	Correcta	No demuestra por sí sola la aceptación funcional en producción.
Prueba de navegador	Recorridos reportados	Auth/API simuladas; no sustituye las pruebas remotas con Supabase real.

8. HALLAZGOS Y ACCIONES PENDIENTES

N.º	Hallazgo	Acción recomendada	Responsable sugerido
H-01	La migración 004 es necesaria para que la vista de base de datos exponga el descuento decimal.	Verificar qué migraciones están aplicadas; aplicar 004 una sola vez en el orden documentado y guardar evidencia.	Renzo
H-02	La publicación, subida de imagen y edición no se validaron con un proyecto Supabase alojado en este análisis.	Ejecutar los casos con cuentas de prueba comprador/vendedor y registrar resultado esperado/obtenido.	Renzo / Brant
H-03	No hay evidencia tabulada de responsive para todo el rango de 320–2560 px.	Probar 320, 375, 768, 1024, 1440 y 2560 px; adjuntar capturas sin datos personales reales.	Brant
H-04	El Definition of Done pide evidencia guardada y ausencia de defectos críticos.	Adjuntar reporte de pruebas, commit/PR y lista de defectos revisada antes de cerrar el sprint.	Ambos

9. CONCLUSIONES

- RF-03 (publicar producto) está implementado en el código: el formulario captura la información necesaria y el backend exige vendedor autorizado, valida los datos y registra la oferta. La aceptación remota de persistencia, fotografía y disponibilidad en catálogo aún debe demostrarse.
- RF-04 (descuento) presenta una corrección relevante respecto del hallazgo inicial: el frontend calcula el porcentaje con un decimal y la migración 004 adapta la vista SQL. El criterio queda respaldado por pruebas locales registradas, pero debe confirmarse que la migración esté aplicada en la base usada por el equipo.
- RF-19 (editar producto) cuenta con control de propiedad, campos permitidos y detección de conflictos, además de pruebas automatizadas relacionadas. La comprobación final debe incluir el intento de edición por parte de un vendedor distinto.
- RNF-06 (responsive) tiene soporte visible en las clases de diseño, pero no puede darse por completamente aceptado sin capturas y mediciones que cubran los viewports comprometidos. Por tanto, el Sprint 2 muestra avance funcional significativo y trazabilidad técnica, con aceptación remota y evidencia visual aún pendientes.

10. BIBLIOGRAFÍA Y FUENTES TÉCNICAS

- Grupo G01. (2026). Planificación de Sprints: Product Backlog y Sprint Backlog. ORBIKA, versión 1.0, 12 de octubre de 2026.
- Grupo G01. (2026). G01_ORBIKA_FD03_SRS_CHATA_LOYOLA. Especificación de requisitos de software (SRS).
- Grupo G01. (2026). G01_ORBIKA_FD04_SAD_CHATA_LOYOLA. Documento de arquitectura de software (SAD).
- Repositorio ORBIKA. (2026). orbika-app/docs/sprints-01-02.md y orbika-app/docs/estado-implementacion.md.
- Repositorio ORBIKA. (2026). Código fuente y pruebas: frontend/src/pages/product.helpers.js; frontend/src/pages/product.helpers.test.js; frontend/src/components/ProductForm.jsx; frontend/src/pages/PublishProduct.jsx; frontend/src/pages/EditProduct.jsx; backend/src/services/producto.service.js; backend/src/validation/producto.validation.js; backend/test/validation/producto.validation.test.js; backend/test/services/producto.service.test.js; supabase/migrations/004_discount_precision.sql.

Repositorio del proyecto: https://github.com/renzoloyola/G01_CSW_ORBIKA